

Hands-On: Migração do Banco de Dados para Data Warehouse

Projeto: Refatoração do Banco 'imp' para Data Warehouse

Data: 25/02/2026

Servidor: 10.209.59.96 (MySQL 8.4)

Ferramenta de acesso: DBeaver / phpMyAdmin (<http://10.209.59.96:8080>)

Índice

1. [Sobre este Hands-On](#)
 2. [Configuração do Ambiente](#)
 3. [Exercício 1 — Diagnóstico do Banco Original](#)
 4. [Exercício 2 — Limpeza de Dados](#)
 5. [Exercício 3 — Padronização de Nomenclatura](#)
 6. [Exercício 4 — Análise dos Dados da Tabela Fato](#)
 7. [Exercício 5 — Migração e Normalização](#)
 8. [Exercício 6 — Chaves Primárias e Estrangeiras](#)
 9. [Exercício 7 — Validação Final e Consultas](#)
 10. [Exercício 8 — Regiões de Trabalho por Órgão](#)
 11. [Referência Rápida](#)
-

1. Sobre este Hands-On

Objetivo

Cada participante irá executar, no seu banco individual, o processo completo de transformação de um banco relacional convencional em um **Data Warehouse** com modelo estrela (Star Schema), passando por:

- Diagnóstico de problemas
- Limpeza de dados
- Padronização de nomenclatura
- Normalização de tabela fato
- Criação de integridade referencial

Estrutura dos Bancos

O banco original **imp** contém os dados de referência. Cada colaborador possui um banco próprio para trabalhar sem impactar os demais:

Colaborador	Banco de Trabalho	Usuário	Senha
Bruna	bruna	bruna	b123456
Lorena	lorena	lorena	l123456

Colaborador	Banco de Trabalho	Usuário	Senha
Rejane	rejane	rejane	r123456
Santino	santino	santino	s123456

(adicionar conforme necessário)

 **IMPORTANTE:** Preencha a tabela acima com os nomes reais e credenciais antes de distribuir.

Pré-Requisitos

- Acesso de rede ao servidor 10.209.59.96 (porta 3306)
- Python 3.10+ instalado na estação de trabalho
- DBeaver ou outro client MySQL instalado
- Git (para clonar o repositório)

Tempo Estimado

Exercício	Duração	Nível
Configuração do Ambiente	30 min	 Básico
Exercício 1 — Diagnóstico	45 min	 Básico
Exercício 2 — Limpeza de Dados	1h 30min	 Intermediário
Exercício 3 — Nomenclatura	1h	 Intermediário
Exercício 4 — Análise de Dados	1h	 Intermediário
Exercício 5 — Migração	1h 30min	 Avançado
Exercício 6 — PKs e FKs	1h	 Intermediário
Exercício 7 — Validação Final	45 min	 Básico
Exercício 8 — Regiões por Órgão	1h 30min	 Avançado
Total	~9h 30min	

2. Configuração do Ambiente

2.1 — Configuração do DBeaver

1. Abra o DBeaver
2. Clique em **Nova Conexão** → **MySQL**
3. Preencha:

Campo	Valor
Host	10.209.59.96

Campo	Valor
Porta	3306
Database	imp_colabX (seu banco)
Usuário	colabX (seu usuário)
Senha	(sua senha)

4. Na aba **Driver Properties**, configure:

Propriedade	Valor
useSSL	false
allowPublicKeyRetrieval	true

5. Clique em **Testar Conexão** → deve retornar "Conectado"

6. Clique em **Concluir**

2.2 — Configuração do Python

Abra o terminal na pasta do projeto e execute:

```
# Clonar o repositório (ou copiar a pasta do projeto)
cd D:\AmbienteDeTrabalho\python\bde

# Criar ambiente virtual
python -m venv venv

# Ativar ambiente virtual
# Windows PowerShell:
.\venv\Scripts\Activate.ps1
# Windows CMD:
.\venv\Scripts\activate.bat
# Linux/Mac:
source venv/bin/activate

# Instalar dependência
pip install mysql-connector-python
```

2.3 — Configurar Conexão nos Scripts Python

Crie ou edite o arquivo `conexao_mysql.py` com **seus dados de acesso**:

```
import mysql.connector
from mysql.connector import Error

# =====
```

```

# ⚠️ ALTERE PARA SEUS DADOS DE ACESSO
# =====
CONFIG = {
    'host': '10.209.59.96',
    'user': 'colabX',          # ← seu usuário
    'password': 'sua_senha',  # ← sua senha
    'database': 'colabX'      # ← seu banco
}

def conectar():
    """Conecta ao banco de dados MySQL"""
    try:
        conexao = mysql.connector.connect(**CONFIG)
        if conexao.is_connected():
            info = conexao.get_server_info()
            cursor = conexao.cursor()
            cursor.execute("SELECT DATABASE();")
            banco = cursor.fetchone()[0]
            print(f"✅ Conectado ao MySQL {info} | Banco: {banco}")
            cursor.close()
            return conexao
    except Error as e:
        print(f"❌ Erro ao conectar: {e}")
        return None

def executar_query(conexao, query, retornar=True):
    """Executa uma query e retorna os resultados"""
    try:
        cursor = conexao.cursor()
        cursor.execute(query)
        if retornar:
            colunas = [desc[0] for desc in cursor.description]
            resultados = cursor.fetchall()
            cursor.close()
            return colunas, resultados
        else:
            conexao.commit()
            afetados = cursor.rowcount
            cursor.close()
            return afetados
    except Error as e:
        print(f"❌ Erro: {e}")
        return None

def fechar(conexao):
    """Fecha a conexão"""
    if conexao and conexao.is_connected():
        conexao.close()
        print("✅ Conexão encerrada")

# Teste rápido
if __name__ == "__main__":
    conn = conectar()
    if conn:

```

```
colunas, tabelas = executar_query(conn, "SHOW TABLES;")
print(f"\n📦 Tabelas encontradas: {len(tabelas)}")
for t in tabelas:
    print(f"    • {t[0]}")
fechar(conn)
```

2.4 — Testar a Conexão

```
python conexao_mysql.py
```

Resultado esperado:

- ☒ Conectado ao MySQL 8.4.x | Banco: colabX
- ☒ Tabelas encontradas: 23
 - tb_aspecto
 - tb_base_cart
 - ...
- ☒ Conexão encerrada

☒ Checkpoint — Ambiente Pronto

Antes de prosseguir, confirme:

- ☐ DBeaver conecta ao banco sem erros
- ☐ Python com `mysql-connector-python` instalado
- ☐ Script `conexao_mysql.py` retorna as 23 tabelas
- ☐ Você consegue visualizar os dados via phpMyAdmin ou DBeaver

3. Exercício 1 — Diagnóstico do Banco Original

Objetivo: Entender o estado atual do banco e identificar problemas

3.1 — Mapear a Estrutura

Execute as queries abaixo no DBeaver ou via Python. **Anote os resultados.**

```
-- 1. Quantas tabelas existem?
SELECT COUNT(*) AS total_tabelas
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE();

-- 2. Quantos registros em cada tabela?
SELECT
    TABLE_NAME AS tabela,
    TABLE_ROWS AS registros,
```

```
ROUND(DATA_LENGTH / 1024 / 1024, 2) AS tamanho_mb
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE()
ORDER BY TABLE_ROWS DESC;

-- 3. Quantas PKs existem?
SELECT COUNT(*) AS total_pks
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = DATABASE()
  AND CONSTRAINT_TYPE = 'PRIMARY KEY';

-- 4. Quantas FKs existem?
SELECT COUNT(*) AS total_fks
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = DATABASE()
  AND CONSTRAINT_TYPE = 'FOREIGN KEY';
```

3.2 — Identificar Problemas de Nomenclatura

```
-- 5. Listar todas as colunas de todas as tabelas
SELECT
  TABLE_NAME AS tabela,
  COLUMN_NAME AS coluna,
  DATA_TYPE AS tipo,
  CHARACTER_MAXIMUM_LENGTH AS tamanho
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = DATABASE()
ORDER BY TABLE_NAME, ORDINAL_POSITION;
```

Anote no seu caderno:

- Quais padrões de prefixo você encontra? (**cod_**, **nome_**, **loc_**, **var_**, **fnt_**, etc.)
- Quais colunas fogem do padrão? (case misto, sem prefixo, abreviações diferentes)
- Quantas tabelas não têm PK? Quantas não têm FK?

3.3 — Identificar Duplicatas

```
-- 6. Duplicatas em tb_base_cart
SELECT cod_base, COUNT(*) AS qtd
FROM tb_base_cart
GROUP BY cod_base
HAVING COUNT(*) > 1;

-- 7. Duplicatas em tb_loc_pai
SELECT loc_cod, COUNT(*) AS qtd
FROM tb_loc_pai
GROUP BY loc_cod
HAVING COUNT(*) > 1;
```

3.4 — Identificar Registros Órfãos

```
-- 8. Localidades em tb_dados que não existem em tb_localidade
SELECT DISTINCT d.loc_cod
FROM tb_dados d
LEFT JOIN tb_localidade l ON d.loc_cod = l.loc_cod
WHERE l.loc_cod IS NULL;

-- 9. Variáveis em tb_dados que não existem em tb_variavel
SELECT DISTINCT d.var_cod
FROM tb_dados d
LEFT JOIN tb_variavel v ON d.var_cod = v.var_cod
WHERE v.var_cod IS NULL;

-- 10. Variáveis em tb_rel_var_fnt que não existem em tb_variavel
SELECT DISTINCT r.var_cod
FROM tb_rel_var_fnt r
LEFT JOIN tb_variavel v ON r.var_cod = v.var_cod
WHERE v.var_cod IS NULL;
```

Entregável do Exercício 1

Preencha a tabela abaixo com seus achados:

Item	Valor Encontrado
Total de tabelas	
Total de registros (todas as tabelas)	
Total de PKs	
Total de FKs	
Tabelas com duplicatas em PK potencial	
Total de registros órfãos em tb_dados (loc_cod)	
Total de registros órfãos em tb_dados (var_cod)	
Principais padrões de nomenclatura identificados	

4. Exercício 2 — Limpeza de Dados

Objetivo: Resolver duplicatas e registros órfãos para possibilitar PKs e FKs

4.1 — Analisar Duplicatas em tb_base_cart

```
-- Ver os registros duplicados em detalhe
SELECT *
```

```
FROM tb_base_cart
WHERE cod_base IN (
    SELECT cod_base
    FROM tb_base_cart
    GROUP BY cod_base
    HAVING COUNT(*) > 1
)
ORDER BY cod_base;
```

 **Pergunta:** Os registros duplicados são idênticos ou possuem dados diferentes?

4.2 — Resolver Duplicatas em tb_base_cart

Escolha **uma** das estratégias abaixo:

Estratégia A — Remover duplicatas idênticas (manter 1):

```
-- Criar tabela temporária com registros únicos
CREATE TABLE tb_base_cart_temp AS
SELECT DISTINCT *
FROM tb_base_cart;

-- Substituir a tabela original
DROP TABLE tb_base_cart;
RENAME TABLE tb_base_cart_temp TO tb_base_cart;
```

Estratégia B — Manter o registro com maior ano:

```
-- Criar tabela com o maior ano por cod_base
CREATE TABLE tb_base_cart_temp AS
SELECT b.*
FROM tb_base_cart b
INNER JOIN (
    SELECT cod_base, MAX(ano) AS max_ano
    FROM tb_base_cart
    GROUP BY cod_base
) m ON b.cod_base = m.cod_base AND b.ano = m.max_ano;

DROP TABLE tb_base_cart;
RENAME TABLE tb_base_cart_temp TO tb_base_cart;
```

4.3 — Resolver Duplicatas em tb_loc_pai

```
-- Analisar duplicatas em detalhe
SELECT *
FROM tb_loc_pai
WHERE loc_cod IN (
```



```
SELECT loc_cod
FROM tb_loc_pai
GROUP BY loc_cod
HAVING COUNT(*) > 1
)
ORDER BY loc_cod;

-- Resolver (adaptar conforme análise)
CREATE TABLE tb_loc_pai_temp AS
SELECT DISTINCT *
FROM tb_loc_pai;

DROP TABLE tb_loc_pai;
RENAME TABLE tb_loc_pai_temp TO tb_loc_pai;
```

4.4 — Resolver Registros Órfãos

Escolha **uma** das estratégias para cada caso:

Opção 1 — Criar registro genérico (recomendado):

```
-- Criar localidade "Desconhecida" para referência
INSERT INTO tb_localidade (loc_cod, loc_nome, loc_nivel)
SELECT DISTINCT d.loc_cod, CONCAT('Localidade Desconhecida (', d.loc_cod, ')'), 0
FROM tb_dados d
LEFT JOIN tb_localidade l ON d.loc_cod = l.loc_cod
WHERE l.loc_cod IS NULL;

-- Criar variável "Desconhecida" para referência
INSERT INTO tb_variavel (var_cod, var_nome)
SELECT DISTINCT d.var_cod, CONCAT('Variável Desconhecida (', d.var_cod, ')')
FROM tb_dados d
LEFT JOIN tb_variavel v ON d.var_cod = v.var_cod
WHERE v.var_cod IS NULL;
```

Opção 2 — Remover registros órfãos:

```
-- ⚠ CUIDADO: Perda de dados!
DELETE d FROM tb_dados d
LEFT JOIN tb_localidade l ON d.loc_cod = l.loc_cod
WHERE l.loc_cod IS NULL;
```

4.5 — Validar Limpeza

```
-- Reexecutar as queries do Exercício 1 (itens 6 a 10)
-- TODAS devem retornar 0 registros
```

```
-- Verificação rápida
SELECT 'tb_base_cart duplicatas' AS verificacao,
       COUNT(*) - COUNT(DISTINCT cod_base) AS problemas
FROM tb_base_cart
UNION ALL
SELECT 'tb_loc_pai duplicatas',
       COUNT(*) - COUNT(DISTINCT loc_cod)
FROM tb_loc_pai
UNION ALL
SELECT 'tb_dados loc_cod órfãos',
       COUNT(DISTINCT d.loc_cod)
FROM tb_dados d LEFT JOIN tb_localidade l ON d.loc_cod = l.loc_cod
WHERE l.loc_cod IS NULL
UNION ALL
SELECT 'tb_dados var_cod órfãos',
       COUNT(DISTINCT d.var_cod)
FROM tb_dados d LEFT JOIN tb_variavel v ON d.var_cod = v.var_cod
WHERE v.var_cod IS NULL;
```

Resultado esperado: Todos os valores na coluna `problemas` devem ser `0`.

☒ Checkpoint — Dados Limpos

- ☐ Zero duplicatas em `tb_base_cart`
- ☐ Zero duplicatas em `tb_loc_pai`
- ☐ Zero registros órfãos em `tb_dados` (`loc_cod` e `var_cod`)
- ☐ Zero registros órfãos em `tb_rel_var_fnt`

5. Exercício 3 — Padronização de Nomenclatura

Objetivo: Renomear tabelas e colunas seguindo o padrão definido

 Consulte o documento `padrao_nomenclatura.md` para a referência completa do de-para.

5.1 — Entender o Padrão

Prefixo	Tipo	Exemplo
<code>dim_</code>	Dimensão	<code>dim_localidade</code>
<code>fact_</code>	Fato	<code>fact_indicador</code>
<code>rel_</code>	Relacionamento N:N	<code>rel_variavel_fonte</code>
<code>aux_</code>	Auxiliar	<code>aux_localidade_hierarquia</code>
<code>cfg_</code>	Configuração	<code>cfg_consulta</code>
<code>log_</code>	Log/Auditoria	<code>log_busca</code>
Sufixo	Uso	Exemplo
<code>_id</code>	Chave primária / FK	<code>localidade_id</code>

Sufixo	Uso	Exemplo
<code>_nome</code>	Nome descritivo	<code>localidade_nome</code>
<code>_txt</code>	Texto longo	<code>nota_txt</code>
<code>_vlr</code>	Valor numérico	<code>indicador_vlr</code>
<code>_dh</code>	Data/hora	<code>busca_dh</code>
<code>_ind</code>	Indicador (boolean)	<code>mapa_disponivel_ind</code>

5.2 — Renomear Colunas (executar por tabela)

⚠️ Execute uma tabela por vez e valide antes de prosseguir.

Começar pelas dimensões simples:

```
-- dim_aspecto
ALTER TABLE tb_aspecto
  CHANGE COLUMN cod_asp aspecto_id TINYINT UNSIGNED NOT NULL,
  CHANGE COLUMN nome_asp aspecto_nome VARCHAR(100);

-- dim_nota
ALTER TABLE tb_nota
  CHANGE COLUMN nota_cod nota_id SMALLINT UNSIGNED NOT NULL,
  CHANGE COLUMN nota_nome nota_txt TEXT;

-- dim_fonte
ALTER TABLE tb_fonte
  CHANGE COLUMN fnt_cod fonte_id SMALLINT UNSIGNED NOT NULL,
  CHANGE COLUMN fnt_sigla fonte_sigla VARCHAR(30),
  CHANGE COLUMN fnt_nome fonte_nome VARCHAR(250);

-- dim_unidade
ALTER TABLE tb_unidade
  CHANGE COLUMN unid_cod unidade_id TINYINT UNSIGNED NOT NULL,
  CHANGE COLUMN unid_nome unidade_nome VARCHAR(100);

-- dim_territorio
ALTER TABLE tb_rel_ter
  CHANGE COLUMN ter_cod territorio_id SMALLINT UNSIGNED NOT NULL,
  CHANGE COLUMN ter_tipo territorio_tipo VARCHAR(50);
```

Depois as dimensões com mais colunas:

```
-- dim_localidade
ALTER TABLE tb_localidade
  CHANGE COLUMN loc_cod localidade_id SMALLINT UNSIGNED NOT NULL,
  CHANGE COLUMN loc_pai localidade_pai_id SMALLINT UNSIGNED,
  CHANGE COLUMN loc_nome localidade_nome VARCHAR(250),
```

```

CHANGE COLUMN loc_nivel localidade_nivel TINYINT UNSIGNED,
CHANGE COLUMN loc_ordem localidade_ordem SMALLINT UNSIGNED,
CHANGE COLUMN loc_cod_ibge localidade_cod_ibge INT UNSIGNED;

-- dim_variavel (colunas principais)
ALTER TABLE tb_variavel
CHANGE COLUMN var_cod variavel_id SMALLINT UNSIGNED NOT NULL,
CHANGE COLUMN unid_cod unidade_id TINYINT UNSIGNED,
CHANGE COLUMN var_nome variavel_nome VARCHAR(500),
CHANGE COLUMN var_asp aspecto_id TINYINT UNSIGNED;

-- dim_base_cartografica
ALTER TABLE tb_base_cart
CHANGE COLUMN cod_base base_cartografica_id SMALLINT UNSIGNED NOT NULL,
CHANGE COLUMN nome_base base_cartografica_nome VARCHAR(200),
CHANGE COLUMN ano_base_ano SMALLINT UNSIGNED,
CHANGE COLUMN nro_munic municipio_qtd SMALLINT UNSIGNED;

```

Tabelas de relacionamento:

```

-- rel_variavel_fonte
ALTER TABLE tb_rel_var_fnt
CHANGE COLUMN var_cod variavel_id SMALLINT UNSIGNED NOT NULL,
CHANGE COLUMN fnt_cod fonte_id SMALLINT UNSIGNED NOT NULL;

-- rel_variavel_nota
ALTER TABLE tb_rel_var_nota
CHANGE COLUMN var_cod variavel_id SMALLINT UNSIGNED NOT NULL,
CHANGE COLUMN nota_cod nota_id SMALLINT UNSIGNED NOT NULL;

-- rel_territorio_variavel
ALTER TABLE tb_rel_ter_var
CHANGE COLUMN ter_cod territorio_id SMALLINT UNSIGNED NOT NULL,
CHANGE COLUMN var_cod variavel_id SMALLINT UNSIGNED NOT NULL;

-- rel_base_ponto
ALTER TABLE tb_base_cart_ptos
CHANGE COLUMN Cod_loc localidade_id SMALLINT UNSIGNED NOT NULL,
CHANGE COLUMN base base_cartografica_id SMALLINT UNSIGNED NOT NULL,
CHANGE COLUMN PtoX ponto_x DECIMAL(15,6),
CHANGE COLUMN PtoY ponto_y DECIMAL(15,6);

-- aux_localidade_hierarquia
ALTER TABLE tb_loc_pai
CHANGE COLUMN loc_cod localidade_id SMALLINT UNSIGNED NOT NULL,
CHANGE COLUMN loc_pai localidade_pai_id SMALLINT UNSIGNED,
CHANGE COLUMN loc_reg regioao_id SMALLINT UNSIGNED;

```

Tabelas de log:

```
-- log_busca
ALTER TABLE tb_log_busca
  CHANGE COLUMN log_data busca_dh DATETIME,
  CHANGE COLUMN log_busca busca_termo VARCHAR(500);

-- log_erro_movimento
ALTER TABLE tb_erro_mvto
  CHANGE COLUMN seq_mvto movimento_seq INT UNSIGNED,
  CHANGE COLUMN seq_cpo campo_seq INT UNSIGNED,
  CHANGE COLUMN msg_erro erro_msg TEXT;
```

5.3 — Renomear Tabelas

⚠ Execute **somente após** todas as colunas terem sido renomeadas com sucesso.

```
RENAME TABLE
  tb_aspecto TO dim_aspecto,
  tb_nota TO dim_nota,
  tb_fonte TO dim_fonte,
  tb_unidade TO dim_unidade,
  tb_localidade TO dim_localidade,
  tb_variavel TO dim_variavel,
  tb_base_cart TO dim_base_cartografica,
  tb_rel_ter TO dim_territorio,
  tb_dados TO fact_indicador,
  tb_dados_mensal TO fact_indicador_mensal,
  tb_rel_var_fnt TO rel_variavel_fonte,
  tb_rel_var_nota TO rel_variavel_nota,
  tb_rel_ter_var TO rel_territorio_variavel,
  tb_base_cart_ptos TO rel_base_ponto,
  tb_loc_pai TO aux_localidade_hierarquia,
  tb_localidade_historico TO aux_localidade_historico,
  tb_localidade_regiao_planejamento_saude TO aux_localidade_regiao_saude,
  tb_consulta TO cfg_consulta,
  tb_infmun TO aux_info_municipio,
  tb_var_calculado TO cfg_variavel_calculada,
  tb_var_produto TO cfg_variavel_produto,
  tb_log_busca TO log_busca,
  tb_erro_mvto TO log_erro_movimento;
```

5.4 — Validar Nomenclatura

```
-- Verificar que não resta nenhuma tabela com prefixo tb_
SELECT TABLE_NAME
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE()
  AND TABLE_NAME LIKE 'tb_%';
-- Esperado: 0 registros
```

```
-- Listar tabelas agrupadas por tipo
SELECT
  CASE
    WHEN TABLE_NAME LIKE 'dim_%' THEN 'Dimensão'
    WHEN TABLE_NAME LIKE 'fact_%' THEN 'Fato'
    WHEN TABLE_NAME LIKE 'rel_%' THEN 'Relacionamento'
    WHEN TABLE_NAME LIKE 'aux_%' THEN 'Auxiliar'
    WHEN TABLE_NAME LIKE 'cfg_%' THEN 'Configuração'
    WHEN TABLE_NAME LIKE 'log_%' THEN 'Log'
    ELSE '⚠ SEM PREFIXO'
  END AS tipo,
  TABLE_NAME AS tabela
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE()
ORDER BY tipo, TABLE_NAME;
```

☑ Checkpoint — Nomenclatura Padronizada

- ☐ Zero tabelas com prefixo **tb_**
- ☐ Todas as colunas em **snake_case**
- ☐ Prefixos corretos por tipo de tabela
- ☐ Query de validação retorna 0 tabelas sem prefixo

6. Exercício 4 — Análise dos Dados da Tabela Fato

Objetivo: Analisar os formatos de dados nas colunas **d_1980** a **d_2030** antes de normalizar

⚠ Este exercício é **crítico**. Uma migração sem análise prévia resulta em **perda de dados**.

6.1 — Descobrir os Formatos

```
-- Contar tipos de valores em d_2020 (ano com mais dados)
SELECT
  CASE
    WHEN d_2020 IS NULL OR TRIM(d_2020) = '' THEN 'VAZIO'
    WHEN TRIM(d_2020) = '-' THEN 'HIFEN (NULL)'
    WHEN TRIM(d_2020) REGEXP '^-[0-9]+$' THEN 'INTEIRO'
    WHEN TRIM(d_2020) REGEXP '^-[0-9]+,[0-9]+$' THEN 'DECIMAL_VIRGULA'
    WHEN TRIM(d_2020) REGEXP '^-[0-9]+\.[0-9]+$' THEN 'DECIMAL_PONTO'
    WHEN TRIM(d_2020) REGEXP '^-[0-9]{1,3}(\.[0-9]{3})+,[0-9]+$' THEN
      'FORMATO_BR_COMPLETO'
    WHEN TRIM(d_2020) REGEXP '^-[0-9]{1,3}(\.[0-9]{3})+$' THEN
      'INTEIRO_COM_MILHARES'
    WHEN TRIM(d_2020) REGEXP '[a-zA-Z]' THEN 'TEXTO'
    ELSE 'OUTRO'
  END AS tipo_dado,
  COUNT(*) AS quantidade,
  ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM fact_indicador), 2) AS
  percentual
```

```
FROM fact_indicador
GROUP BY tipo_dado
ORDER BY quantidade DESC;
```

6.2 — Ver Exemplos de Cada Tipo

```
-- Exemplos de valores com vírgula (formato BR)
SELECT DISTINCT d_2020
FROM fact_indicador
WHERE d_2020 REGEXP '^-[0-9]+,[0-9]+$'
LIMIT 10;

-- Exemplos de valores com ponto e vírgula (BR completo)
SELECT DISTINCT d_2020
FROM fact_indicador
WHERE d_2020 REGEXP '^-[0-9]{1,3}(\.[0-9]{3})+,[0-9]+$'
LIMIT 10;

-- Exemplos de texto/caracteres especiais
SELECT DISTINCT d_2020
FROM fact_indicador
WHERE d_2020 REGEXP '[a-zA-Z]' OR d_2020 LIKE '%[%]'
LIMIT 10;

-- Exemplos de valores com espaços
SELECT CONCAT(' ', d_2020, ' ') AS valor_com_aspas, LENGTH(d_2020) AS tamanho
FROM fact_indicador
WHERE d_2020 LIKE ' %' OR d_2020 LIKE '% '
LIMIT 10;
```

6.3 — Usando o Script Python

Execute o script de análise automática:

```
python analisar_dados_migracao.py
```

💡 Lembre-se de ajustar a configuração de conexão no script antes de executar.

📄 Entregável do Exercício 4

Tipo de Dado	Quantidade	%	Exemplo
INTEIRO			
HIFEN (NULL)			
DECIMAL_VIRGULA			

Tipo de Dado	Quantidade	%	Exemplo
DECIMAL_PONTO			
FORMATO_BR_COMPLETO			
INTEIRO_COM_MILHARES			
TEXTO			
OUTRO			

 **Pergunta para reflexão:** Por que **não** podemos simplesmente fazer `ALTER TABLE ... MODIFY COLUMN d_2020 DECIMAL(20,6)?`

7. Exercício 5 — Migração e Normalização

Objetivo: Criar a estrutura normalizada e migrar os dados com conversão de formatos

7.1 — Criar Dimensão Tempo

```
-- Criar tabela dim_tempo (granularidade anual + mensal)
CREATE TABLE dim_tempo (
    tempo_id INT AUTO_INCREMENT PRIMARY KEY,
    ano SMALLINT NOT NULL COMMENT 'Ano (1980-2030)',
    mes TINYINT NULL COMMENT 'Mês (1-12), NULL = registro anual',
    trimestre TINYINT NULL COMMENT 'Trimestre (1-4), NULL = registro anual',
    semestre TINYINT NULL COMMENT 'Semestre (1-2), NULL = registro anual',
    decada VARCHAR(10) NOT NULL COMMENT 'Década',
    seculo TINYINT NOT NULL COMMENT 'Século',
    ano_bissexto_ind BOOLEAN NOT NULL DEFAULT FALSE COMMENT 'É ano bissexto?',
    descricao VARCHAR(50) NOT NULL COMMENT 'Descrição do período',
    UNIQUE KEY uk_ano_mes (ano, mes),
    INDEX idx_decada (decada),
    INDEX idx_ano (ano),
    INDEX idx_mes (mes)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Etapa 1: Registros anuais (mes = NULL) – compatibilidade com dados existentes
INSERT INTO dim_tempo (ano, mes, trimestre, semestre, decada, seculo,
ano_bissexto_ind, descricao)
WITH RECURSIVE anos AS (
    SELECT 1980 AS ano
    UNION ALL
    SELECT ano + 1 FROM anos WHERE ano < 2030
)
SELECT
    ano,
    NULL, NULL, NULL,
    CONCAT(FLOOR(ano/10)*10, 's'),
    CASE WHEN ano < 2000 THEN 20 ELSE 21 END,
    (ano % 4 = 0 AND (ano % 100 != 0 OR ano % 400 = 0)),
```



```

        CONCAT('Ano ', ano)
FROM anos;

-- Etapa 2: Registros mensais (12 por ano) – granularidade expandida
INSERT INTO dim_tempo (ano, mes, trimestre, semestre, decada, seculo,
ano_bissesto_ind, descricao)
WITH RECURSIVE anos AS (
    SELECT 1980 AS ano
    UNION ALL
    SELECT ano + 1 FROM anos WHERE ano < 2030
),
meses AS (
    SELECT 1 AS mes UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT 4
    UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8
    UNION ALL SELECT 9 UNION ALL SELECT 10 UNION ALL SELECT 11 UNION ALL SELECT 12
)
SELECT
    a.ano, m.mes,
    CEIL(m.mes / 3),
    CEIL(m.mes / 6),
    CONCAT(FLOOR(a.ano/10)*10, 's'),
    CASE WHEN a.ano < 2000 THEN 20 ELSE 21 END,
    (a.ano % 4 = 0 AND (a.ano % 100 != 0 OR a.ano % 400 = 0)),
    CONCAT(
        CASE m.mes
            WHEN 1 THEN 'Jan' WHEN 2 THEN 'Fev' WHEN 3 THEN 'Mar'
            WHEN 4 THEN 'Abr' WHEN 5 THEN 'Mai' WHEN 6 THEN 'Jun'
            WHEN 7 THEN 'Jul' WHEN 8 THEN 'Ago' WHEN 9 THEN 'Set'
            WHEN 10 THEN 'Out' WHEN 11 THEN 'Nov' WHEN 12 THEN 'Dez'
        END, '/', a.ano
    )
FROM anos a
CROSS JOIN meses m;

-- Validar
SELECT mes IS NULL AS tipo, COUNT(*) AS qtd FROM dim_tempo GROUP BY (mes IS NULL);
-- Esperado: 51 registros anuais (mes IS NULL) + 612 mensais = 663 total

SELECT * FROM dim_tempo WHERE ano = 2020 ORDER BY mes;
-- Esperado: 13 registros (1 anual + 12 mensais)

```

💡 **Por que dois níveis de granularidade?** O banco atual (`fact_indicador_original`) armazena dados **apenas por ano** (colunas `d_1980` a `d_2030`). Os registros anuais (`mes IS NULL`) garantem compatibilidade com a migração atual. Os registros mensais preparam a estrutura para futuras cargas de dados com granularidade mensal — por exemplo, dados da tabela `fact_indicador_mensal` que hoje está vazia.

7.2 — Criar a Função de Conversão

```
DELIMITER $$
```

```

DROP FUNCTION IF EXISTS fn_converter_valor_numerico$$

CREATE FUNCTION fn_converter_valor_numerico(valor_original VARCHAR(100))
RETURNS DECIMAL(20,6)
DETERMINISTIC
BEGIN
    DECLARE valor_limpo VARCHAR(100);
    DECLARE valor_final DECIMAL(20,6);

    SET valor_limpo = TRIM(valor_original);

    -- Nulo ou vazio → NULL
    IF valor_limpo IS NULL OR valor_limpo = '' THEN
        RETURN NULL;
    END IF;

    -- Hífen → NULL
    IF valor_limpo = '-' THEN
        RETURN NULL;
    END IF;

    -- Colchetes e texto → NULL
    IF valor_limpo LIKE '%[%]' OR valor_limpo REGEXP '[a-zA-Z]' THEN
        RETURN NULL;
    END IF;

    -- Formato BR completo: 1.234.567,89
    IF valor_limpo REGEXP '^-[0-9]{1,3}(\.[0-9]{3})+,[0-9]+$' THEN
        SET valor_limpo = REPLACE(valor_limpo, '.', '');
        SET valor_limpo = REPLACE(valor_limpo, ',', '.');
    -- Formato BR decimal: 0,56
    ELSEIF valor_limpo REGEXP '^-[0-9]+,[0-9]+$' THEN
        SET valor_limpo = REPLACE(valor_limpo, ',', '.');
    -- Inteiro com ponto de milhares: 1.234.567
    ELSEIF valor_limpo REGEXP '^-[0-9]{1,3}(\.[0-9]{3})+$' THEN
        SET valor_limpo = REPLACE(valor_limpo, '.', '');
    END IF;

    -- Tentar converter
    BEGIN
        DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
            SET valor_final = NULL;
        SET valor_final = CAST(valor_limpo AS DECIMAL(20,6));
    END;

    RETURN valor_final;
END$$

DELIMITER ;

```

7.3 — Testar a Função

```
-- Teste de todos os formatos encontrados
SELECT
    valor_teste,
    fn_converter_valor_numerico(valor_teste) AS convertido
FROM (
    SELECT '12345' AS valor_teste
    UNION ALL SELECT '-'
    UNION ALL SELECT '0,56'
    UNION ALL SELECT '1.234.567,89'
    UNION ALL SELECT ' 385.623.865,48 '
    UNION ALL SELECT '1.067.267'
    UNION ALL SELECT '-0,23'
    UNION ALL SELECT '[1]'
    UNION ALL SELECT 'x'
    UNION ALL SELECT ''
    UNION ALL SELECT NULL
) testes;
```

Resultado esperado:

valor_teste	convertido
12345	12345.000000
-	NULL
0,56	0.560000
1.234.567,89	1234567.890000
385.623.865,48	385623865.480000
1.067.267	1067267.000000
-0,23	-0.230000
[1]	NULL
x	NULL
(vazio)	NULL
NULL	NULL

 **Se algum resultado não bater**, revise a função antes de prosseguir!

7.4 — Criar Tabela Fato

```
CREATE TABLE fact_indicador (
    indicador_id BIGINT AUTO_INCREMENT PRIMARY KEY,

    -- Chaves estrangeiras
    localidade_id SMALLINT UNSIGNED NOT NULL,
```

```

variavel_id SMALLINT UNSIGNED NOT NULL,
tempo_id INT NOT NULL,

-- Valores
indicador_txt VARCHAR(100) NULL
    COMMENT 'Valor original preservado',
indicador_vlr DECIMAL(20,6) NULL
    COMMENT 'Valor convertido para número',
indicador_tipo ENUM('numero','texto','nulo_hifen','vazio','texto_especial')
NOT NULL
    COMMENT 'Classificação do dado original',

-- Controle
carga_dh TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

-- Índices
UNIQUE KEY uk_indicador (localidade_id, variavel_id, tempo_id),
KEY idx_tempo (tempo_id),
KEY idx_tipo (indicador_tipo)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

💡 **Observe:** A tabela original `fact_indicador` (ex-`tb_dados`) será renomeada para `fact_indicador_original` ao final, e esta nova tabela assumirá o nome `fact_indicador`.

7.5 — Migrar os Dados (1 ano para teste)

```

-- Migrar apenas d_2020 como teste
INSERT INTO fact_indicador
    (localidade_id, variavel_id, tempo_id, indicador_txt, indicador_vlr,
    indicador_tipo)
SELECT
    f.localidade_id,
    f.variavel_id,
    t.tempo_id,
    f.d_2020 AS indicador_txt,
    fn_converter_valor_numerico(f.d_2020) AS indicador_vlr,
    CASE
        WHEN TRIM(f.d_2020) IS NULL OR TRIM(f.d_2020) = '' THEN 'vazio'
        WHEN TRIM(f.d_2020) = '-' THEN 'nulo_hifen'
        WHEN TRIM(f.d_2020) REGEXP '[a-zA-Z]' THEN 'texto'
        WHEN TRIM(f.d_2020) LIKE '%[%]' THEN 'texto_especial'
        ELSE 'numero'
    END AS indicador_tipo
FROM fact_indicador_original f
JOIN dim_tempo t ON t.ano = 2020 AND t.mes IS NULL
WHERE f.d_2020 IS NOT NULL;

-- Validar
SELECT
    indicador_tipo,
    COUNT(*) AS qtd,

```

```
ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM fact_indicador), 2) AS pct
FROM fact_indicador
GROUP BY indicador_tipo
ORDER BY qtd DESC;
```

7.6 — Migrar Todos os Anos

⚠ Esta etapa pode demorar **10-30 minutos** dependendo do hardware.

```
-- Limpar a tabela de teste
TRUNCATE TABLE fact_indicador;

-- Executar migração completa via procedure
-- (ver arquivo funcao_conversao_dados.sql)
CALL sp_migrar_dados_para_fato();
```

Ou manualmente, repetindo o INSERT da seção 7.5 para cada ano:

```
-- Gerar os INSERTs para todos os anos
-- Pode usar um loop ou executar individualmente
-- Exemplo para d_2019:
INSERT INTO fact_indicador
    (localidade_id, variavel_id, tempo_id, indicador_txt, indicador_vlr,
    indicador_tipo)
SELECT
    f.localidade_id, f.variavel_id, t.tempo_id,
    f.d_2019, fn_converter_valor_numerico(f.d_2019),
    CASE
        WHEN TRIM(f.d_2019) IS NULL OR TRIM(f.d_2019) = '' THEN 'vazio'
        WHEN TRIM(f.d_2019) = '-' THEN 'nulo_hifen'
        WHEN TRIM(f.d_2019) REGEXP '[a-zA-Z]' THEN 'texto'
        WHEN TRIM(f.d_2019) LIKE '%[%]' THEN 'texto_especial'
        ELSE 'numero'
    END
    FROM fact_indicador_original f
    JOIN dim_tempo t ON t.ano = 2019 AND t.mes IS NULL
    WHERE f.d_2019 IS NOT NULL;
-- Repetir para cada ano de 1980 a 2030...
```

7.7 — Validar Migração Completa

```
-- 1. Total de registros migrados
SELECT COUNT(*) AS total FROM fact_indicador;
-- Esperado: ~3.032.785

-- 2. Distribuição de tipos
SELECT
```

```
    indicador_tipo,
    COUNT(*) AS qtd,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM fact_indicador), 2) AS pct
FROM fact_indicador
GROUP BY indicador_tipo
ORDER BY qtd DESC;

-- 3. Verificar se há números que não converteram
SELECT COUNT(*) AS numeros_sem_conversao
FROM fact_indicador
WHERE indicador_tipo = 'numero' AND indicador_vlr IS NULL;
-- Esperado: 0

-- 4. Renomear tabela original como backup
RENAME TABLE fact_indicador_original TO fact_indicador_backup_colunar;
```

☑ Checkpoint — Migração Concluída

- ☐ dim_tempo criada com 663 registros (51 anuais + 612 mensais)
- ☐ Função de conversão testada com todos os formatos
- ☐ fact_indicador populada com dados de todos os anos
- ☐ Distribuição de tipos compatível com a análise
- ☐ Zero números sem conversão
- ☐ Tabela original preservada como backup

8. Exercício 6 — Chaves Primárias e Estrangeiras

Objetivo: Implementar integridade referencial no Data Warehouse

8.1 — Criar Chaves Primárias

```
-- Dimensões simples
ALTER TABLE dim_aspecto ADD PRIMARY KEY (aspecto_id);
ALTER TABLE dim_nota ADD PRIMARY KEY (nota_id);
ALTER TABLE dim_fonte ADD PRIMARY KEY (fonte_id);
ALTER TABLE dim_unidade ADD PRIMARY KEY (unidade_id);
ALTER TABLE dim_territorio ADD PRIMARY KEY (territorio_id);

-- Dimensões compostas
ALTER TABLE dim_localidade ADD PRIMARY KEY (localidade_id);
ALTER TABLE dim_variavel ADD PRIMARY KEY (variavel_id);
ALTER TABLE dim_base_cartografica ADD PRIMARY KEY (base_cartografica_id);

-- Relacionamentos (PKs compostas)
ALTER TABLE rel_variavel_fonte ADD PRIMARY KEY (variavel_id, fonte_id);
ALTER TABLE rel_variavel_nota ADD PRIMARY KEY (variavel_id, nota_id);
ALTER TABLE rel_territorio_variavel ADD PRIMARY KEY (territorio_id, variavel_id);
ALTER TABLE rel_base_ponto ADD PRIMARY KEY (localidade_id, base_cartografica_id);
```

```
-- Auxiliares
ALTER TABLE aux_localidade_hierarquia ADD PRIMARY KEY (localidade_id);
```

8.2 — Validar PKs

```
SELECT
    TABLE_NAME AS tabela,
    CONSTRAINT_NAME AS pk
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = DATABASE()
    AND CONSTRAINT_TYPE = 'PRIMARY KEY'
ORDER BY TABLE_NAME;
-- Esperado: Mínimo 13 PKs
```

8.3 — Criar Chaves Estrangeiras

```
-- dim_variavel → dim_unidade
ALTER TABLE dim_variavel
    ADD CONSTRAINT fk_variavel_unidade
    FOREIGN KEY (unidade_id) REFERENCES dim_unidade(unidade_id);

-- dim_variavel → dim_aspecto
ALTER TABLE dim_variavel
    ADD CONSTRAINT fk_variavel_aspecto
    FOREIGN KEY (aspecto_id) REFERENCES dim_aspecto(aspecto_id);

-- fact_indicador → dim_localidade
ALTER TABLE fact_indicador
    ADD CONSTRAINT fk_fact_localidade
    FOREIGN KEY (localidade_id) REFERENCES dim_localidade(localidade_id);

-- fact_indicador → dim_variavel
ALTER TABLE fact_indicador
    ADD CONSTRAINT fk_fact_variavel
    FOREIGN KEY (variavel_id) REFERENCES dim_variavel(variavel_id);

-- fact_indicador → dim_tempo
ALTER TABLE fact_indicador
    ADD CONSTRAINT fk_fact_tempo
    FOREIGN KEY (tempo_id) REFERENCES dim_tempo(tempo_id);

-- rel_variavel_fonte → dim_variavel / dim_fonte
ALTER TABLE rel_variavel_fonte
    ADD CONSTRAINT fk_rel_vf_variavel
    FOREIGN KEY (variavel_id) REFERENCES dim_variavel(variavel_id),
    ADD CONSTRAINT fk_rel_vf_fonte
    FOREIGN KEY (fonte_id) REFERENCES dim_fonte(fonte_id);

-- rel_variavel_nota → dim_variavel / dim_nota
```

```

ALTER TABLE rel_variavel_nota
  ADD CONSTRAINT fk_rel_vn_variavel
  FOREIGN KEY (variavel_id) REFERENCES dim_variavel(variavel_id),
  ADD CONSTRAINT fk_rel_vn_nota
  FOREIGN KEY (nota_id) REFERENCES dim_nota(nota_id);

-- rel_territorio_variavel → dim_territorio / dim_variavel
ALTER TABLE rel_territorio_variavel
  ADD CONSTRAINT fk_rel_tv_territorio
  FOREIGN KEY (territorio_id) REFERENCES dim_territorio(territorio_id),
  ADD CONSTRAINT fk_rel_tv_variavel
  FOREIGN KEY (variavel_id) REFERENCES dim_variavel(variavel_id);

-- rel_base_ponto → dim_localidade / dim_base_cartografica
ALTER TABLE rel_base_ponto
  ADD CONSTRAINT fk_rel_bp_localidade
  FOREIGN KEY (localidade_id) REFERENCES dim_localidade(localidade_id),
  ADD CONSTRAINT fk_rel_bp_base
  FOREIGN KEY (base_cartografica_id) REFERENCES
dim_base_cartografica(base_cartografica_id);

```

8.4 — Validar FKs

```

SELECT
  TABLE_NAME AS tabela,
  CONSTRAINT_NAME AS fk,
  REFERENCED_TABLE_NAME AS referencia
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = DATABASE()
  AND REFERENCED_TABLE_NAME IS NOT NULL
ORDER BY TABLE_NAME;
-- Esperado: Mínimo 11 FKs

-- Testar integridade: inserir registro órfão (deve falhar)
INSERT INTO fact_indicador (localidade_id, variavel_id, tempo_id, indicador_tipo)
VALUES (99999, 1, 1, 'numero');
-- Esperado: ERROR 1452 - Cannot add or update a child row

```

☒ Checkpoint — Integridade Implementada

- ☐ Mínimo 13 PKs criadas
- ☐ Mínimo 11 FKs criadas
- ☐ Teste de integridade falha corretamente ao inserir órfão
- ☐ Nenhum erro ao criar PKs/FKs (se houver, resolver dados antes)

9. Exercício 7 — Validação Final e Consultas

Objetivo: Validar o Data Warehouse completo e testar consultas típicas

9.1 — Validação Completa

```
-- Resumo do Data Warehouse
SELECT
    CASE
        WHEN TABLE_NAME LIKE 'dim_%' THEN '📊 Dimensão'
        WHEN TABLE_NAME LIKE 'fact_%' THEN '📈 Fato'
        WHEN TABLE_NAME LIKE 'rel_%' THEN '🔗 Relacionamento'
        WHEN TABLE_NAME LIKE 'aux_%' THEN '🔧 Auxiliar'
        WHEN TABLE_NAME LIKE 'cfg_%' THEN '⚙️ Configuração'
        WHEN TABLE_NAME LIKE 'log_%' THEN '📄 Log'
    END AS tipo,
    TABLE_NAME AS tabela,
    TABLE_ROWS AS registros
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE()
    AND TABLE_NAME NOT LIKE '%backup%'
ORDER BY tipo, TABLE_NAME;

-- Contar constraints
SELECT
    CONSTRAINT_TYPE AS tipo,
    COUNT(*) AS quantidade
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = DATABASE()
GROUP BY CONSTRAINT_TYPE;
```

9.2 — Consultas Típicas de Data Warehouse

Consulta 1 — Indicadores de uma localidade em um ano:

```
SELECT
    l.localidade_nome,
    v.variavel_nome,
    u.unidade_nome,
    f.indicador_vlr,
    t.ano
FROM fact_indicador f
JOIN dim_localidade l ON f.localidade_id = l.localidade_id
JOIN dim_variavel v ON f.variavel_id = v.variavel_id
JOIN dim_unidade u ON v.unidade_id = u.unidade_id
JOIN dim_tempo t ON f.tempo_id = t.tempo_id
WHERE l.localidade_nome LIKE '%Rio de Janeiro%'
    AND t.ano = 2020
    AND f.indicador_tipo = 'numero'
ORDER BY v.variavel_nome
LIMIT 20;
```

Consulta 2 — Evolução de um indicador ao longo dos anos:

```
SELECT
    t.ano,
    f.indicador_vlr
FROM fact_indicador f
JOIN dim_tempo t ON f.tempo_id = t.tempo_id
JOIN dim_variavel v ON f.variavel_id = v.variavel_id
WHERE v.variavel_nome LIKE '%população%'
      AND f.localidade_id = 33 -- Ajustar conforme localidade
      AND f.indicador_tipo = 'numero'
ORDER BY t.ano;
```

Consulta 3 — Ranking de localidades por indicador:

```
SELECT
    l.localidade_nome,
    f.indicador_vlr
FROM fact_indicador f
JOIN dim_localidade l ON f.localidade_id = l.localidade_id
JOIN dim_variavel v ON f.variavel_id = v.variavel_id
JOIN dim_tempo t ON f.tempo_id = t.tempo_id
WHERE v.variavel_id = 100 -- Ajustar conforme variável
      AND t.ano = 2020
      AND f.indicador_tipo = 'numero'
ORDER BY f.indicador_vlr DESC
LIMIT 10;
```

Consulta 4 — Fontes de uma variável:

```
SELECT
    v.variavel_nome,
    fo.fonte_sigla,
    fo.fonte_nome
FROM rel_variavel_fonte rf
JOIN dim_variavel v ON rf.variavel_id = v.variavel_id
JOIN dim_fonte fo ON rf.fonte_id = fo.fonte_id
WHERE v.variavel_id = 100;
```

9.3 — Comparação Antes x Depois

```
-- ANTES (estrutura antiga – consulta para referência)
-- SELECT l.loc_nome, v.var_nome, d.d_2020
-- FROM tb_dados d
-- JOIN tb_localidade l ON d.loc_cod = l.loc_cod
-- JOIN tb_variavel v ON d.var_cod = v.var_cod
-- WHERE l.loc_nivel = 3;
```

```
-- DEPOIS (estrutura nova)
SELECT
    l.localidade_nome,
    v.variavel_nome,
    f.indicador_vlr
FROM fact_indicador f
JOIN dim_localidade l ON f.localidade_id = l.localidade_id
JOIN dim_variavel v ON f.variavel_id = v.variavel_id
JOIN dim_tempo t ON f.tempo_id = t.tempo_id
WHERE l.localidade_nivel = 3
      AND t.ano = 2020
      AND f.indicador_tipo = 'numero';
```

 Entregável Final

Critério	Status
Banco sem tabelas com prefixo <code>tb_</code>	☐
Todas as colunas em <code>snake_case</code>	☐
Zero duplicatas em chaves primárias	☐
Zero registros órfãos	☐
dim_tempo com 663 registros (51 anuais + 612 mensais)	☐
fact_indicador com ~3M registros	☐
Mínimo 13 PKs criadas	☐
Mínimo 11 FKs criadas	☐
Consultas de teste executam < 2s	☐
Tabela original preservada como backup	☐

10. Exercício 8 — Regiões de Trabalho por Órgão

Objetivo: Implementar o modelo Snowflake parcial com SCD Tipo 2 para suportar múltiplas regionalizações por órgão com rastreabilidade temporal

Duração estimada: 1h 30min

Nível:  Avançado

 Documentação de referência: [MODELO_REGIOES_TRABALHO.md](#)

10.1 — Entender a Problemática

No Estado de Goiás, **cada órgão define suas próprias regiões de trabalho**. Um mesmo município pertence simultaneamente a regiões de órgãos diferentes, e essas vinculações **mudam ao longo do tempo**:

Órgão	Tipo de Regionalização	Exemplo
-------	------------------------	---------

Órgão	Tipo de Regionalização	Exemplo
Estado de Goiás	Regiões de Planejamento	Metropolitana de Goiânia, Entorno do DF
Secretaria de Saúde	Macro e Microrregiões	Centro-Oeste, Goiânia, Inhumas
Secretaria de Segurança	RISP	RISP Capital, RISP Anápolis
Secretaria de Educação	Subsecretarias Regionais	Subsecretaria de Anápolis

Perguntas para reflexão:

- O modelo estrela atual suporta adicionar a regionalização da Secretaria de Educação **sem ALTER TABLE**?
- Se um município mudou de microrregião de saúde em 2023, como consultamos a qual microrregião ele pertencia em 2019?

10.2 — Criar as Novas Tabelas

Execute as DDLs abaixo. As tabelas já estão incluídas na PARTE 7 do [script_padronizacao_nomenclatura.sql](#), mas vamos executar passo a passo para entender cada uma.

Passo 1 — **dim_orgao** (Órgãos que definem regionalizações):

```
CREATE TABLE IF NOT EXISTS dim_orgao (
  orgao_id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT
    COMMENT 'PK - Surrogate key',
  orgao_nome VARCHAR(200) NOT NULL
    COMMENT 'Nome completo do órgão',
  orgao_sigla VARCHAR(20) NULL
    COMMENT 'Sigla do órgão',
  orgao_nivel ENUM('estadual', 'secretaria', 'autarquia', 'federal',
'municipal')
    NOT NULL DEFAULT 'secretaria'
    COMMENT 'Nível institucional',
  ativo_ind BOOLEAN NOT NULL DEFAULT TRUE
    COMMENT 'Indica se o órgão está ativo',
  carga_dh TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizacao_dh TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

  PRIMARY KEY (orgao_id),
  UNIQUE KEY uk_orgao_nome (orgao_nome),
  INDEX idx_orgao_sigla (orgao_sigla)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='Dimensão Órgão - Órgãos que definem regionalizações de trabalho';
```

Observe:

- **orgao_nivel** usa **ENUM** — limitando valores válidos no nível do banco
- **UNIQUE KEY uk_orgao_nome** — impede duplicação de órgãos

Passo 2 — dim_regiao (Regiões de trabalho genéricas, com hierarquia):

```

CREATE TABLE IF NOT EXISTS dim_regiao (
    regiao_id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT
        COMMENT 'PK - Surrogate key',
    orgao_id SMALLINT UNSIGNED NOT NULL
        COMMENT 'FK - Órgão dono desta regionalização',
    regiao_pai_id SMALLINT UNSIGNED NULL
        COMMENT 'FK - Região pai (auto-referência para multi-nível)',
    regiao_nome VARCHAR(200) NOT NULL
        COMMENT 'Nome da região',
    regiao_sigla VARCHAR(20) NULL,
    regiao_nivel TINYINT UNSIGNED NOT NULL DEFAULT 1
        COMMENT 'Nível hierárquico (1=macro, 2=micro, 3=local)',
    regiao_tipo VARCHAR(50) NULL
        COMMENT 'Tipo descritivo (Região de Planejamento, Macrorregião, etc.)',
    regiao_cod_externo VARCHAR(20) NULL
        COMMENT 'Código em sistema externo (IBGE, SUS, etc.)',
    ativa_ind BOOLEAN NOT NULL DEFAULT TRUE,
    carga_dh TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizacao_dh TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    PRIMARY KEY (regiao_id),
    INDEX idx_regiao_orgao (orgao_id),
    INDEX idx_regiao_pai (regiao_pai_id),
    INDEX idx_regiao_tipo (regiao_tipo),

    CONSTRAINT fk_regiao_orgao FOREIGN KEY (orgao_id)
        REFERENCES dim_orgao (orgao_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_regiao_pai FOREIGN KEY (regiao_pai_id)
        REFERENCES dim_regiao (regiao_id) ON UPDATE CASCADE ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='Dimensão Região - Regiões de trabalho com hierarquia multi-nível';

```

📝 Observe:

- **regiao_pai_id** é uma **auto-referência** — permite hierarquia Macro → Micro → Local
- **regiao_cod_externo** — guarda o código do sistema de origem (ex.: código SUS da macrorregião)
- A FK **fk_regiao_orgao** garante que toda região pertence a um órgão válido

Passo 3 — bridge_localidade_regiao (Vínculo com temporalidade SCD Tipo 2):

```

CREATE TABLE IF NOT EXISTS bridge_localidade_regiao (
    vinculo_id INT UNSIGNED NOT NULL AUTO_INCREMENT
        COMMENT 'PK - Surrogate key',
    localidade_id SMALLINT UNSIGNED NOT NULL
        COMMENT 'FK - Localidade (município)',
    regiao_id SMALLINT UNSIGNED NOT NULL
        COMMENT 'FK - Região de trabalho',

```

```

vigencia_inicio_dt DATE NOT NULL
    COMMENT 'Data de início da vigência',
vigencia_fim_dt DATE NULL
    COMMENT 'Data de fim da vigência (NULL = vigente)',
vigente_ind BOOLEAN GENERATED ALWAYS AS (vigencia_fim_dt IS NULL) STORED
    COMMENT 'Flag de conveniência: TRUE se vigente',
motivo_alteracao_txt VARCHAR(500) NULL
    COMMENT 'Motivo da mudança de região',
norma_legal_txt VARCHAR(200) NULL
    COMMENT 'Decreto/Lei/Portaria que definiu a mudança',
carga_dh TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

PRIMARY KEY (vinculo_id),
INDEX idx_bridge_localidade (localidade_id),
INDEX idx_bridge_regiao (regiao_id),
INDEX idx_bridge_vigente (vigente_ind),
INDEX idx_bridge_vigencia (vigencia_inicio_dt, vigencia_fim_dt),
UNIQUE KEY uk_bridge_vigencia (localidade_id, regiao_id, vigencia_inicio_dt),

CONSTRAINT fk_bridge_localidade FOREIGN KEY (localidade_id)
    REFERENCES dim_localidade (localidade_id) ON UPDATE CASCADE ON DELETE
RESTRICT,
CONSTRAINT fk_bridge_regiao FOREIGN KEY (regiao_id)
    REFERENCES dim_regiao (regiao_id) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='Bridge Table - Vínculo localidade↔região com temporalidade (SCD Tipo 2)';

```

Conceitos importantes:

- **vigente_ind** é uma **coluna GENERATED** — calculada automaticamente pelo MySQL
- **vigencia_fim_dt = NULL** significa **vigente**; com data preenchida significa **encerrado**
- A **UNIQUE KEY** impede que a mesma localidade entre na mesma região duas vezes na mesma data

10.3 — Validar a Estrutura

```

-- Verificar que as 3 tabelas foram criadas
SELECT TABLE_NAME, TABLE_COMMENT
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE()
    AND TABLE_NAME IN ('dim_orgao', 'dim_regiao', 'bridge_localidade_regiao');
-- Esperado: 3 linhas

-- Verificar a coluna GENERATED
SHOW COLUMNS FROM bridge_localidade_regiao WHERE Field = 'vigente_ind';
-- Esperado: Extra = STORED GENERATED

-- Verificar FKs criadas
SELECT CONSTRAINT_NAME, TABLE_NAME, REFERENCED_TABLE_NAME
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = DATABASE()
    AND TABLE_NAME IN ('dim_regiao', 'bridge_localidade_regiao')

```

```

    AND REFERENCED_TABLE_NAME IS NOT NULL;
-- Esperado: 4 FKs (fk_regiao_orgao, fk_regiao_pai, fk_bridge_localidade,
fk_bridge_regiao)

```

10.4 — Popular com Dados Reais

 O script completo está em `migrar_dados_regioes.sql`. Aqui executamos passo a passo.

Passo 1 — Inserir órgãos:

```

INSERT INTO dim_orgao (orgao_nome, orgao_sigla, orgao_nivel) VALUES
    ('Estado de Goiás', 'GO', 'estadual'),
    ('Secretaria de Estado da Saúde', 'SES-GO', 'secretaria');
-- SSP, Educação e outros podem ser adicionados depois

-- Validar
SELECT * FROM dim_orgao;
-- Esperado: 2 registros

```

Passo 2 — Migrar Regiões de Planejamento (Estado):

```

-- Extrair regiões de planejamento distintas da dim_localidade
INSERT INTO dim_regiao (orgao_id, regiao_nome, regiao_nivel, regiao_tipo,
regiao_cod_externo)
SELECT DISTINCT
    (SELECT orgao_id FROM dim_orgao WHERE orgao_sigla = 'GO'),
    CONCAT('Região de Planejamento ', regiao_planejamento_id),
    1,
    'Região de Planejamento',
    CAST(regiao_planejamento_id AS CHAR)
FROM dim_localidade
WHERE regiao_planejamento_id IS NOT NULL;

-- Validar
SELECT * FROM dim_regiao WHERE regiao_tipo = 'Região de Planejamento';

```

 **Observe:** Usamos `regiao_cod_externo` para guardar o ID original da `dim_localidade` — será usado no próximo passo para fazer a ligação na bridge.

Passo 3 — Migrar Regiões de Saúde (Macro e Micro):

```

-- 3a. Macrorregiões de Saúde (nível 1)
INSERT INTO dim_regiao (orgao_id, regiao_nome, regiao_nivel, regiao_tipo,
regiao_cod_externo)
SELECT DISTINCT
    (SELECT orgao_id FROM dim_orgao WHERE orgao_sigla = 'SES-GO'),
    regiao_macro_saude_nome,

```

```

1,
'Macrorregião de Saúde',
CAST(regiao_macro_saude_id AS CHAR)
FROM aux_localidade_regiao_saude
WHERE regiao_macro_saude_nome IS NOT NULL
AND TRIM(regiao_macro_saude_nome) != '';

-- 3b. Microrregiões de Saúde (nível 2, com pai = macrorregião)
INSERT INTO dim_regiao (orgao_id, regiao_pai_id, regiao_nome, regiao_nivel,
regiao_tipo, regiao_cod_externo)
SELECT DISTINCT
(SELECT orgao_id FROM dim_orgao WHERE orgao_sigla = 'SES-GO'),
r_macro.regiao_id,
ars.regiao_micro_saude_nome,
2,
'Microrregião de Saúde',
CAST(ars.regiao_micro_saude_id AS CHAR)
FROM aux_localidade_regiao_saude ars
JOIN dim_regiao r_macro
ON r_macro.regiao_cod_externo = CAST(ars.regiao_macro_saude_id AS CHAR)
AND r_macro.regiao_tipo = 'Macrorregião de Saúde'
WHERE ars.regiao_micro_saude_nome IS NOT NULL
AND TRIM(ars.regiao_micro_saude_nome) != '';

-- Validar hierarquia
SELECT
r.regiao_tipo,
r.regiao_nome,
rp.regiao_nome AS regiao_pai
FROM dim_regiao r
LEFT JOIN dim_regiao rp ON r.regiao_pai_id = rp.regiao_id
WHERE r.orgao_id = (SELECT orgao_id FROM dim_orgao WHERE orgao_sigla = 'SES-GO')
ORDER BY r.regiao_nivel, r.regiao_nome;

```

 **Pergunta para reflexão:** O que acontece no JOIN da microrregião se houver uma macrorregião_id na tabela de saúde que não foi inserida no passo 3a? (Dica: o JOIN impede o INSERT — protege a integridade)

Passo 4 — Popular a Bridge com vínculos atuais:

```

-- 4a. Vínculos de Região de Planejamento
INSERT INTO bridge_localidade_regiao
(localidade_id, regiao_id, vigencia_inicio_dt, motivo_alteracao_txt)
SELECT
l.localidade_id,
r.regiao_id,
'2000-01-01',
'Carga inicial - migração do modelo anterior'
FROM dim_localidade l
JOIN dim_regiao r
ON r.regiao_cod_externo = CAST(l.regiao_planejamento_id AS CHAR)
AND r.regiao_tipo = 'Região de Planejamento'

```



```

WHERE l.regiao_planejamento_id IS NOT NULL;

-- 4b. Vínculos de Macrorregião de Saúde
INSERT INTO bridge_localidade_regiao
    (localidade_id, regiao_id, vigencia_inicio_dt, motivo_alteracao_txt)
SELECT
    l.localidade_id,
    r.regiao_id,
    '2000-01-01',
    'Carga inicial - migração do modelo anterior'
FROM dim_localidade l
JOIN aux_localidade_regiao_saude ars
    ON l.localidade_cod_ibge = ars.localidade_cod_ibge
JOIN dim_regiao r
    ON r.regiao_cod_externo = CAST(ars.regiao_macro_saude_id AS CHAR)
    AND r.regiao_tipo = 'Macrorregião de Saúde'
WHERE ars.regiao_macro_saude_id IS NOT NULL;

-- 4c. Vínculos de Microrregião de Saúde
INSERT INTO bridge_localidade_regiao
    (localidade_id, regiao_id, vigencia_inicio_dt, motivo_alteracao_txt)
SELECT
    l.localidade_id,
    r.regiao_id,
    '2000-01-01',
    'Carga inicial - migração do modelo anterior'
FROM dim_localidade l
JOIN aux_localidade_regiao_saude ars
    ON l.localidade_cod_ibge = ars.localidade_cod_ibge
JOIN dim_regiao r
    ON r.regiao_cod_externo = CAST(ars.regiao_micro_saude_id AS CHAR)
    AND r.regiao_tipo = 'Microrregião de Saúde'
WHERE ars.regiao_micro_saude_id IS NOT NULL;

-- Validar
SELECT
    r.regiao_tipo,
    COUNT(*) AS vinculos,
    SUM(b.vigente_ind) AS vigentes
FROM bridge_localidade_regiao b
JOIN dim_regiao r ON b.regiao_id = r.regiao_id
GROUP BY r.regiao_tipo
ORDER BY r.regiao_tipo;

```

10.5 — Criar Procedure de Movimentação

```

DELIMITER $$

DROP PROCEDURE IF EXISTS sp_mover_localidade_regiao$$

CREATE PROCEDURE sp_mover_localidade_regiao(

```

```

    IN p_localidade_id SMALLINT UNSIGNED,
    IN p_nova_regiao_id SMALLINT UNSIGNED,
    IN p_data_mudanca DATE,
    IN p_motivo VARCHAR(500),
    IN p_norma_legal VARCHAR(200)
)
BEGIN
    DECLARE v_orgao_id SMALLINT UNSIGNED;
    DECLARE v_nivel TINYINT UNSIGNED;
    DECLARE v_vinculo_anterior INT UNSIGNED;

    -- 1. Obter órgão e nível da nova região
    SELECT orgao_id, regiao_nivel
    INTO v_orgao_id, v_nivel
    FROM dim_regiao
    WHERE regiao_id = p_nova_regiao_id;

    -- 2. Encontrar vínculo vigente anterior (mesmo órgão e nível)
    SELECT b.vinculo_id
    INTO v_vinculo_anterior
    FROM bridge_localidade_regiao b
    JOIN dim_regiao r ON b.regiao_id = r.regiao_id
    WHERE b.localidade_id = p_localidade_id
        AND r.orgao_id = v_orgao_id
        AND r.regiao_nivel = v_nivel
        AND b.vigente_ind = TRUE
    LIMIT 1;

    -- 3. Encerrar vínculo anterior (se existir)
    IF v_vinculo_anterior IS NOT NULL THEN
        UPDATE bridge_localidade_regiao
        SET vigencia_fim_dt = DATE_SUB(p_data_mudanca, INTERVAL 1 DAY)
        WHERE vinculo_id = v_vinculo_anterior;
    END IF;

    -- 4. Criar novo vínculo
    INSERT INTO bridge_localidade_regiao
    (localidade_id, regiao_id, vigencia_inicio_dt,
    motivo_alteracao_txt, norma_legal_txt)
    VALUES
    (p_localidade_id, p_nova_regiao_id, p_data_mudanca,
    p_motivo, p_norma_legal);

    SELECT CONCAT('✅ Localidade ', p_localidade_id,
        ' movida para região ', p_nova_regiao_id,
        ' a partir de ', p_data_mudanca) AS resultado;

END$$

DELIMITER ;

```

Passo a passo da procedure:

1. Descubra a qual órgão e nível pertence a nova região

2. Procura se já existe um vínculo vigente para aquele município, no mesmo órgão e nível
3. Se existe, **encerra** o vínculo anterior (preenche `vigencia_fim_dt`)
4. Cria o **novo vínculo** com `vigencia_inicio_dt` = data da mudança

10.6 — Testar a Movimentação

Vamos simular que o município de Itaberaí (ajuste o `localidade_id` conforme seu banco) mudou de microrregião de saúde em 2023:

```
-- 1. Consultar situação atual de um município
SELECT
    l.localidade_nome,
    o.orgao_sigla,
    r.regiao_tipo,
    r.regiao_nome,
    b.vigencia_inicio_dt,
    CASE WHEN b.vigente_ind THEN '☑ Vigente' ELSE '☐ Encerrado' END AS status
FROM bridge_localidade_regiao b
JOIN dim_localidade l ON b.localidade_id = l.localidade_id
JOIN dim_regiao r ON b.regiao_id = r.regiao_id
JOIN dim_orgao o ON r.orgao_id = o.orgao_id
WHERE l.localidade_nome LIKE '%Itabera%'
ORDER BY o.orgao_sigla, r.regiao_nivel;

-- 2. Obter o regiao_id da nova microrregião (Inhumas)
SELECT regiao_id, regiao_nome
FROM dim_regiao
WHERE regiao_nome LIKE '%Inhumas%'
    AND regiao_tipo = 'Microrregião de Saúde';
-- Anotar o regiao_id retornado

-- 3. Executar movimentação
-- ⚠ Substituir os IDs conforme seu banco!
CALL sp_mover_localidade_regiao(
    52, -- localidade_id (Itaberaí)
    @regiao_inhumas_id, -- regiao_id (substituir pelo valor
real)
    '2023-01-01', -- data da mudança
    'Redistribuição sanitário 2023', -- motivo
    'Portaria SES/GO nº 123/2022' -- norma legal
);

-- 4. Consultar o HISTÓRICO após movimentação
SELECT
    l.localidade_nome,
    r.regiao_tipo,
    r.regiao_nome,
    b.vigencia_inicio_dt,
    b.vigencia_fim_dt,
    CASE WHEN b.vigente_ind THEN '☑ Vigente' ELSE '☐ Encerrado' END AS status,
    b.motivo_alteracao_txt
FROM bridge_localidade_regiao b
```

```
JOIN dim_localidade l ON b.localidade_id = l.localidade_id
JOIN dim_regiao r ON b.regiao_id = r.regiao_id
WHERE l.localidade_nome LIKE '%Itabera%'
      AND r.regiao_tipo = 'Microrregião de Saúde'
ORDER BY b.vigencia_inicio_dt;
```

Resultado esperado:

localidade_nome	regiao_nome	vigencia_inicio	vigencia_fim	status	motivo
Itaberaí	Goiânia	2000-01-01	2022-12-31	☐ Encerrado	Carga inicial
Itaberaí	Inhumas	2023-01-01	NULL	☒ Vigente	Redistribuição sanitário 2023

10.7 — Criar Views de Conveniência

```
-- View 1: Regiões vigentes (para consultas do dia-a-dia)
CREATE OR REPLACE VIEW vw_localidade_regioes_vigentes AS
SELECT
    l.localidade_id,
    l.localidade_nome,
    l.localidade_cod_ibge,
    o.orgao_id,
    o.orgao_nome,
    o.orgao_sigla,
    r.regiao_id,
    r.regiao_nome,
    r.regiao_nivel,
    r.regiao_tipo,
    b.vigencia_inicio_dt
FROM dim_localidade l
JOIN bridge_localidade_regiao b ON l.localidade_id = b.localidade_id
JOIN dim_regiao r ON b.regiao_id = r.regiao_id
JOIN dim_orgao o ON r.orgao_id = o.orgao_id
WHERE b.vigente_ind = TRUE;

-- View 2: Histórico completo (para consultas por data de referência)
CREATE OR REPLACE VIEW vw_localidade_regioes_historico AS
SELECT
    l.localidade_id,
    l.localidade_nome,
    l.localidade_cod_ibge,
    o.orgao_id,
    o.orgao_nome,
    o.orgao_sigla,
    r.regiao_id,
    r.regiao_nome,
    r.regiao_nivel,
    r.regiao_tipo,
```

```

        b.vigencia_inicio_dt,
        b.vigencia_fim_dt,
        b.vigente_ind,
        b.motivo_alteracao_txt
FROM dim_localidade l
JOIN bridge_localidade_regiao b ON l.localidade_id = b.localidade_id
JOIN dim_regiao r ON b.regiao_id = r.regiao_id
JOIN dim_orgao o ON r.orgao_id = o.orgao_id;

-- Testar view vigentes
SELECT orgao_sigla, regiao_tipo, regiao_nome, COUNT(*) AS municipios
FROM vw_localidade_regioes_vigentes
GROUP BY orgao_sigla, regiao_tipo, regiao_nome
ORDER BY orgao_sigla, regiao_tipo, regiao_nome;

```

10.8 — Consultas Avançadas com o Modelo de Regiões

Consulta 1 — Indicadores agregados por macrorregião de saúde:

```

SELECT
    r.regiao_nome AS macrorregiao,
    COUNT(DISTINCT l.localidade_id) AS qtd_municipios,
    COUNT(f.indicador_id) AS qtd_indicadores,
    SUM(f.indicador_vlr) AS total,
    AVG(f.indicador_vlr) AS media
FROM fact_indicador f
JOIN dim_localidade l ON f.localidade_id = l.localidade_id
JOIN dim_tempo t ON f.tempo_id = t.tempo_id
JOIN bridge_localidade_regiao b ON l.localidade_id = b.localidade_id
JOIN dim_regiao r ON b.regiao_id = r.regiao_id
JOIN dim_orgao o ON r.orgao_id = o.orgao_id
WHERE t.ano = 2020
    AND o.orgao_sigla = 'SES-GO'
    AND r.regiao_tipo = 'Macrorregião de Saúde'
    AND f.indicador_tipo = 'numero'
    AND '2020-06-15' BETWEEN b.vigencia_inicio_dt
        AND COALESCE(b.vigencia_fim_dt, '9999-12-31')
GROUP BY r.regiao_nome
ORDER BY total DESC;

```

Consulta 2 — Municípios de uma microrregião em uma data específica:

```

-- Quais municípios pertenciam à Microrregião "Goiânia" em junho de 2019?
SELECT
    l.localidade_nome,
    l.localidade_cod_ibge
FROM bridge_localidade_regiao b
JOIN dim_localidade l ON b.localidade_id = l.localidade_id
JOIN dim_regiao r ON b.regiao_id = r.regiao_id

```

```
WHERE r.regiao_nome = 'Goiânia'
AND r.regiao_tipo = 'Microrregião de Saúde'
AND '2019-06-15' BETWEEN b.vigencia_inicio_dt
AND COALESCE(b.vigencia_fim_dt, '9999-12-31')
ORDER BY l.localidade_nome;
```

Consulta 3 — Comparar composição de uma região em dois momentos:

```
-- Municípios que SAÍRAM de uma microrregião entre 2019 e 2024
SELECT
    l.localidade_nome,
    b.vigencia_inicio_dt,
    b.vigencia_fim_dt,
    b.motivo_alteracao_txt
FROM bridge_localidade_regiao b
JOIN dim_localidade l ON b.localidade_id = l.localidade_id
JOIN dim_regiao r ON b.regiao_id = r.regiao_id
WHERE r.regiao_nome = 'Goiânia'
AND r.regiao_tipo = 'Microrregião de Saúde'
AND b.vigencia_fim_dt BETWEEN '2019-01-01' AND '2024-12-31'
ORDER BY b.vigencia_fim_dt;
```

10.9 — Validação com Python

Execute o script de validação para verificar o modelo de regiões:

```
python validar_refatoracao.py
# Escolha a opção 5: "Validar modelo de regiões por órgão"
```

10.10 — Limpeza (Opcional — apenas após validação completa)

⚠ SOMENTE execute esta etapa se toda a migração foi validada com sucesso e os vínculos na bridge estão corretos.

```
-- Remover colunas de região fixa da dim_localidade
-- (os dados agora vivem na bridge_localidade_regiao)
ALTER TABLE dim_localidade
    DROP COLUMN regioao_planejamento_id,
    DROP COLUMN regioao_macro_saude_id,
    DROP COLUMN regioao_micro_saude_id;
```

☒ Checkpoint — Modelo de Regiões Implementado

- ☐ 3 novas tabelas criadas (`dim_orgao`, `dim_regiao`, `bridge_localidade_regiao`)
- ☐ Órgãos cadastrados (mínimo 2: GO e SES)

- ☐ Regiões de planejamento migradas
- ☐ Macrorregiões e microrregiões de saúde migradas com hierarquia
- ☐ Bridge populada com vínculos vigentes
- ☐ Procedure `sp_mover_localidade_regiao` testada
- ☐ Teste de movimentação gera histórico correto (SCD Tipo 2)
- ☐ Views `vw_localidade_regioes_vigentes` e `vw_localidade_regioes_historico` funcionando
- ☐ Consulta de indicadores por região retorna dados consistentes

Entregável do Exercício 8

Critério	Status
<code>dim_orgao</code> com ≥ 2 órgãos	☐
<code>dim_regiao</code> com regiões de planejamento + saúde (macro e micro)	☐
<code>bridge_localidade_regiao</code> com vínculos vigentes	☐
Hierarquia Macro→Micro funcionando (<code>regiao_pai_id</code>)	☐
Movimentação via procedure testada	☐
Histórico com SCD Tipo 2 (encerrado + vigente)	☐
Views criadas e retornando dados	☐
Consulta por data de referência funciona	☐

11. Referência Rápida

Arquivos do Projeto

Arquivo	Descrição
<code>conexao_mysql.py</code>	Script de conexão ao banco
<code>validar_refatoracao.py</code>	Validação automática da estrutura
<code>analisar_dados_migracao.py</code>	Análise de tipos de dados nas colunas d_YYYY
<code>funcao_conversao_dados.sql</code>	Funções SQL + procedure de migração
<code>script_padronizacao_nomenclatura.sql</code>	Script completo de renomeação + criação de estruturas
<code>migrar_dados_regioes.sql</code>	Migração de dados para o modelo de regiões por órgão
<code>padrao_nomenclatura.md</code>	Documentação do padrão de nomenclatura
<code>RELATORIO_ANALISE_DADOS.md</code>	Relatório detalhado da análise de dados
<code>CRONOGRAMA_IMPLEMENTACAO.md</code>	Cronograma completo do projeto
<code>MODELO_REGIOES_TRABALHO.md</code>	Evolução do modelo: regiões de trabalho por órgão (Snowflake + SCD Tipo 2)

Modelo Estrela com Snowflake Parcial (Regiões)

```
erDiagram
    dim_aspecto {
        int aspecto_id PK
        varchar aspecto_nome
    }

    dim_unidade {
        smallint unidade_id PK
        varchar unidade_nome
    }

    dim_fonte {
        smallint fonte_id PK
        varchar fonte_sigla
        varchar fonte_nome
    }

    dim_nota {
        smallint nota_id PK
        text nota_txt
    }

    dim_variavel {
        smallint variavel_id PK
        varchar variavel_nome
        smallint unidade_id FK
        int aspecto_id FK
        year variavel_ultimo_ano
    }

    dim_localidade {
        smallint localidade_id PK
        varchar localidade_nome
        tinyint localidade_nivel
        varchar localidade_cod_ibge
        smallint localidade_pai_id FK
    }

    dim_tempo {
        int tempo_id PK
        year ano
        tinyint mes
        tinyint trimestre
        tinyint semestre
        varchar decada
        tinyint seculo
        boolean ano_bissesto_ind
        varchar descricao
    }
```



```

fact_indicador {
    bigint indicador_id PK
    smallint localidade_id FK
    smallint variavel_id FK
    int tempo_id FK
    decimal indicador_vlr
    varchar indicador_txt
    enum indicador_tipo
}

dim_orgao {
    smallint orgao_id PK
    varchar orgao_nome
    varchar orgao_sigla
    enum orgao_nivel
    boolean ativo_ind
}

dim_regiao {
    smallint regiao_id PK
    smallint orgao_id FK
    smallint regiao_pai_id FK
    varchar regiao_nome
    tinyint regiao_nivel
    varchar regiao_tipo
    varchar regiao_cod_externo
}

bridge_localidade_regiao {
    int vinculo_id PK
    smallint localidade_id FK
    smallint regiao_id FK
    date vigencia_inicio_dt
    date vigencia_fim_dt
    boolean vigente_ind
    varchar motivo_alteracao_txt
    varchar norma_legal_txt
}

rel_variavel_fonte {
    smallint variavel_id FK
    smallint fonte_id FK
}

rel_variavel_nota {
    smallint variavel_id FK
    smallint nota_id FK
}

%% — Modelo Estrela
dim_variavel }o--|| dim_aspecto      : "aspecto_id"
dim_variavel }o--|| dim_unidade      : "unidade_id"
fact_indicador }o--|| dim_variavel    : "variavel_id"
fact_indicador }o--|| dim_localidade  : "localidade_id"

```

```

fact_indicador }o--|| dim_tempo      : "tempo_id"

%% — Relacionamentos N:N —————
rel_variavel_fonte }o--|| dim_variavel : "variavel_id"
rel_variavel_fonte }o--|| dim_fonte    : "fonte_id"
rel_variavel_nota   }o--|| dim_variavel : "variavel_id"
rel_variavel_nota   }o--|| dim_nota     : "nota_id"

%% — Snowflake Parcial: Regiões (SCD Tipo 2) —————
dim_orgao           ||--o{ dim_regiao      : "define regiões"
dim_regiao           }o--o| dim_regiao      : "pai (multi-nível)"
dim_localidade       ||--o{ bridge_localidade_regiao : "pertence a"
dim_regiao           ||--o{ bridge_localidade_regiao : "agrupa"

```

Comandos Úteis

```

-- Ver estrutura de uma tabela
DESCRIBE dim_localidade;

-- Ver constraints de uma tabela
SELECT * FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'fact_indicador';

-- Contar registros rapidamente
SELECT TABLE_NAME, TABLE_ROWS
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE()
ORDER BY TABLE_ROWS DESC;

-- Ver índices
SHOW INDEX FROM fact_indicador;

```

Contato para Dúvidas

Em caso de problemas durante o hands-on:

1. Verifique se a conexão com o servidor **10.209.59.96** está ativa
2. Confirme que está usando o banco correto (**SELECT DATABASE();**)
3. Consulte os documentos de referência na pasta do projeto
4. Chame o instrutor

Bom trabalho! 🚀